

4. Metaclasses and Attributes

Metaclasses are often mentioned in lists of Python's features, but few understand what they accomplish in practice. The name *metaclass* vaguely implies a concept above and beyond a class. Simply put, metaclasses let you intercept Python's `class` statement and provide special behavior each time a class is defined.

Similarly mysterious and powerful are Python's built-in features for dynamically customizing attribute accesses. Along with Python's object-oriented constructs, these facilities provide wonderful tools to ease the transition from simple classes to complex ones.

However, with these powers come many pitfalls. Dynamic attributes enable you to override objects and cause unexpected side effects. Metaclasses can create extremely bizarre behaviors that are unapproachable to newcomers. It's important that you follow the *rule of least surprise* and only use these mechanisms to implement well-understood idioms.

Item 29: Use Plain Attributes Instead of Get and Set Methods

Programmers coming to Python from other languages may naturally try to implement explicit getter and setter methods in their classes.

```
class OldResistor(object):
    def __init__(self, ohms):
        self._ohms = ohms

    def get_ohms(self):
        return self._ohms

    def set_ohms(self, ohms):
        self._ohms = ohms
```

Using these setters and getters is simple, but it's not Pythonic.

[Click here to view code image](#)

```
r0 = OldResistor(50e3)
print('Before: %5r' % r0.get_ohms())
r0.set_ohms(10e3)
print('After: %5r' % r0.get_ohms())

>>>
Before: 50000.0
After: 10000.0
```

Such methods are especially clumsy for operations like incrementing in place.

[Click here to view code image](#)

```
r0.set_ohms(r0.get_ohms() + 5e3)
```

These utility methods do help define the interface for your class, making it easier to encapsulate functionality, validate usage, and define boundaries. Those are important

goals when designing a class to ensure you don't break callers as your class evolves over time.

In Python, however, you almost never need to implement explicit setter or getter methods. Instead, you should always start your implementations with simple public attributes.

[Click here to view code image](#)

```
class Resistor(object):
    def __init__(self, ohms):
        self.ohms = ohms
        self.voltage = 0
        self.current = 0
```

```
r1 = Resistor(50e3)
r1.ohms = 10e3
```

These make operations like incrementing in place natural and clear.

```
r1.ohms += 5e3
```

Later, if you decide you need special behavior when an attribute is set, you can migrate to the `@property` decorator and its corresponding `setter` attribute. Here, I define a new subclass of `Resistor` that lets me vary the `current` by assigning the `voltage` property. Note that in order to work properly the name of both the setter and getter methods must match the intended property name.

[Click here to view code image](#)

```
class VoltageResistance(Resistor):
    def __init__(self, ohms):
        super().__init__(ohms)
        self._voltage = 0

    @property
    def voltage(self):
        return self._voltage

    @voltage.setter
    def voltage(self, voltage):
        self._voltage = voltage
        self.current = self._voltage / self.ohms
```

Now, assigning the `voltage` property will run the `voltage` setter method, updating the `current` property of the object to match.

[Click here to view code image](#)

```
r2 = VoltageResistance(1e3)
print('Before: %5r amps' % r2.current)
r2.voltage = 10
print('After:  %5r amps' % r2.current)

>>>
Before:      0 amps
After:    0.01 amps
```

Specifying a `setter` on a property also lets you perform type checking and validation on values passed to your class. Here, I define a class that ensures all resistance values are above zero ohms:

[Click here to view code image](#)

```

class BoundedResistance(Resistor):
    def __init__(self, ohms):
        super().__init__(ohms)

    @property
    def ohms(self):
        return self._ohms

    @ohms.setter
    def ohms(self, ohms):
        if ohms <= 0:
            raise ValueError('%f ohms must be > 0' % ohms)
        self._ohms = ohms

```

Assigning an invalid resistance to the attribute raises an exception.

[Click here to view code image](#)

```

r3 = BoundedResistance(1e3)
r3.ohms = 0

>>>
ValueError: 0.000000 ohms must be > 0

```

An exception will also be raised if you pass an invalid value to the constructor.

[Click here to view code image](#)

```

BoundedResistance(-5)
>>>
ValueError: -5.000000 ohms must be > 0

```

This happens because `BoundedResistance.__init__` calls `Resistor.__init__`, which assigns `self.ohms = -5`. That assignment causes the `@ohms.setter` method from `BoundedResistance` to be called, immediately running the validation code before object construction has completed.

You can even use `@property` to make attributes from parent classes immutable.

[Click here to view code image](#)

```

class FixedResistance(Resistor):
    # ...
    @property
    def ohms(self):
        return self._ohms

    @ohms.setter
    def ohms(self, ohms):
        if hasattr(self, '_ohms'):
            raise AttributeError("Can't set attribute")
        self._ohms = ohms

```

Trying to assign to the property after construction raises an exception.

[Click here to view code image](#)

```

r4 = FixedResistance(1e3)
r4.ohms = 2e3

>>>
AttributeError: Can't set attribute

```

The biggest shortcoming of `@property` is that the methods for an attribute can only be shared by subclasses. Unrelated classes can't share the same implementation. However, Python also supports *descriptors* (see [Item 31: “Use Descriptors for Reusable @property Methods”](#)) that enable reusable property logic and many other use cases.

Finally, when you use `@property` methods to implement setters and getters, be sure that the behavior you implement is not surprising. For example, don't set other attributes in getter property methods.

[Click here to view code image](#)

```
class MysteriousResistor(Resistor):
    @property
    def ohms(self):
        self.voltage = self._ohms * self.current
        return self._ohms
    # ...
```

This leads to extremely bizarre behavior.

[Click here to view code image](#)

```
r7 = MysteriousResistor(10)
r7.current = 0.01
print('Before: %5r' % r7.voltage)
r7.ohms
print('After:  %5r' % r7.voltage)

>>>
Before:      0
After:      0.1
```

The best policy is to only modify related object state in `@property.setter` methods. Be sure to avoid any other side effects the caller may not expect beyond the object, such as importing modules dynamically, running slow helper functions, or making expensive database queries. Users of your class will expect its attributes to be like any other Python object: quick and easy. Use normal methods to do anything more complex or slow.

Things to Remember

- ◆ Define new class interfaces using simple public attributes, and avoid set and get methods.
- ◆ Use `@property` to define special behavior when attributes are accessed on your objects, if necessary.
- ◆ Follow the rule of least surprise and avoid weird side effects in your `@property` methods.
- ◆ Ensure that `@property` methods are fast; do slow or complex work using normal methods.

Item 30: Consider @property Instead of Refactoring Attributes

The built-in `@property` decorator makes it easy for simple accesses of an instance's attributes to act smarter (see [Item 29: “Use Plain Attributes Instead of Get and Set Methods”](#)). One advanced but common use of `@property` is transitioning what was once a simple numerical attribute into an on-the-fly calculation. This is extremely helpful because it lets you migrate all existing usage of a class to have new behaviors without rewriting any of the call sites. It also provides an important stopgap for improving your interfaces over time.

For example, say you want to implement a leaky bucket quota using plain Python objects. Here, the `Bucket` class represents how much quota remains and the duration for which the quota will be available:

[Click here to view code image](#)

```
class Bucket(object):
    def __init__(self, period):
        self.period_delta = timedelta(seconds=period)
        self.reset_time = datetime.now()
        self.quota = 0

    def __repr__(self):
        return 'Bucket(quota=%d)' % self.quota
```

The leaky bucket algorithm works by ensuring that, whenever the bucket is filled, the amount of quota does not carry over from one period to the next.

[Click here to view code image](#)

```
def fill(bucket, amount):
    now = datetime.now()
    if now - bucket.reset_time > bucket.period_delta:
        bucket.quota = 0
        bucket.reset_time = now
    bucket.quota += amount
```

Each time a quota consumer wants to do something, it first must ensure that it can deduct the amount of quota it needs to use.

[Click here to view code image](#)

```
def deduct(bucket, amount):
    now = datetime.now()
    if now - bucket.reset_time > bucket.period_delta:
        return False
    if bucket.quota - amount < 0:
        return False
    bucket.quota -= amount
    return True
```

To use this class, first I fill the bucket.

```
bucket = Bucket(60)
fill(bucket, 100)
print(bucket)
```

```
>>>
```

```
Bucket(quota=100)
```

Then, I deduct the quota that I need.

[Click here to view code image](#)

```
if deduct(bucket, 99):
    print('Had 99 quota')
else:
    print('Not enough for 99 quota')
print(bucket)

>>>
Had 99 quota
Bucket(quota=1)
```

Eventually, I'm prevented from making progress because I try to deduct more quota than is available. In this case, the bucket's quota level remains unchanged.

[Click here to view code image](#)

```
if deduct(bucket, 3):
    print('Had 3 quota')
else:
    print('Not enough for 3 quota')
print(bucket)

>>>
Not enough for 3 quota
Bucket(quota=1)
```

The problem with this implementation is that I never know what quota level the bucket started with. The quota is deducted over the course of the period until it reaches zero. At that point, `deduct` will always return `False`. When that happens, it would be useful to know whether callers to `deduct` are being blocked because the `Bucket` ran out of quota or because the `Bucket` never had quota in the first place.

To fix this, I can change the class to keep track of the `max_quota` issued in the period and the `quota_consumed` in the period.

[Click here to view code image](#)

```
class Bucket(object):
    def __init__(self, period):
        self.period_delta = timedelta(seconds=period)
        self.reset_time = datetime.now()
        self.max_quota = 0
        self.quota_consumed = 0

    def __repr__(self):
        return ('Bucket(max_quota=%d, quota_consumed=%d)' %
                (self.max_quota, self.quota_consumed))
```

I use a `@property` method to compute the current level of quota on-the-fly using these new attributes.

[Click here to view code image](#)

```
@property
def quota(self):
    return self.max_quota - self.quota_consumed
```

When the `quota` attribute is assigned, I take special action matching the current interface of the class used by `fill` and `deduct`.

[Click here to view code image](#)

```
@quota.setter
def quota(self, amount):
    delta = self.max_quota - amount
    if amount == 0:
        # Quota being reset for a new period
        self.quota_consumed = 0
        self.max_quota = 0
    elif delta < 0:
        # Quota being filled for the new period
        assert self.quota_consumed == 0
        self.max_quota = amount
    else:
        # Quota being consumed during the period
        assert self.max_quota >= self.quota_consumed
        self.quota_consumed += delta
```

Rerunning the demo code from above produces the same results.

[Click here to view code image](#)

```
bucket = Bucket(60)
print('Initial', bucket)
fill(bucket, 100)
print('Filled', bucket)

if deduct(bucket, 99):
    print('Had 99 quota')
else:
    print('Not enough for 99 quota')

print('Now', bucket)

if deduct(bucket, 3):
    print('Had 3 quota')
else:
    print('Not enough for 3 quota')

print('Still', bucket)

>>>
Initial Bucket(max_quota=0, quota_consumed=0)
Filled Bucket(max_quota=100, quota_consumed=0)
Had 99 quota
Now Bucket(max_quota=100, quota_consumed=99)
Not enough for 3 quota
Still Bucket(max_quota=100, quota_consumed=99)
```

The best part is that the code using `Bucket.quota` doesn't have to change or know that the class has changed. New usage of `Bucket` can do the right thing and access `max_quota` and `quota_consumed` directly.

I especially like `@property` because it lets you make incremental progress toward a better data model over time. Reading the `Bucket` example above, you may have thought to yourself, “`fill` and `deduct` should have been implemented as instance methods in

the first place.” Although you’re probably right (see [Item 22: “Prefer Helper Classes Over Bookkeeping with Dictionaries and Tuples”](#)), in practice there are many situations in which objects start with poorly defined interfaces or act as dumb data containers. This happens when code grows over time, scope increases, multiple authors contribute without anyone considering long-term hygiene, etc.

`@property` is a tool to help you address problems you’ll come across in real-world code. Don’t overuse it. When you find yourself repeatedly extending `@property` methods, it’s probably time to refactor your class instead of further paving over your code’s poor design.

Things to Remember

- ◆ Use `@property` to give existing instance attributes new functionality.
- ◆ Make incremental progress toward better data models by using `@property`.
- ◆ Consider refactoring a class and all call sites when you find yourself using `@property` too heavily.

Item 31: Use Descriptors for Reusable `@property` Methods

The big problem with the `@property` built-in (see [Item 29: “Use Plain Attributes Instead of Get and Set Methods”](#) and [Item 30: “Consider `@property` Instead of Refactoring Attributes”](#)) is reuse. The methods it decorates can’t be reused for multiple attributes of the same class. They also can’t be reused by unrelated classes.

For example, say you want a class to validate that the grade received by a student on a homework assignment is a percentage.

[Click here to view code image](#)

```
class Homework(object):
    def __init__(self):
        self._grade = 0

    @property
    def grade(self):
        return self._grade

    @grade.setter
    def grade(self, value):
        if not (0 <= value <= 100):
            raise ValueError('Grade must be between 0 and 100')
        self._grade = value
```

Using an `@property` makes this class easy to use.

```
galileo = Homework()
galileo.grade = 95
```

Say you also want to give the student a grade for an exam, where the exam has multiple subjects, each with a separate grade.

[Click here to view code image](#)

```
class Exam(object):
```

```

def __init__(self):
    self._writing_grade = 0
    self._math_grade = 0

    @staticmethod
    def _check_grade(value):
        if not (0 <= value <= 100):
            raise ValueError('Grade must be between 0 and 100')

```

This quickly gets tedious. Each section of the exam requires adding a new `@property` and related validation.

[Click here to view code image](#)

```

@property
def writing_grade(self):
    return self._writing_grade

@writing_grade.setter
def writing_grade(self, value):
    self._check_grade(value)
    self._writing_grade = value

@property
def math_grade(self):
    return self._math_grade

@math_grade.setter
def math_grade(self, value):
    self._check_grade(value)
    self._math_grade = value

```

Also, this approach is not general. If you want to reuse this percentage validation beyond homework and exams, you'd need to write the `@property` boilerplate and `_check_grade` repeatedly.

The better way to do this in Python is to use a *descriptor*. The descriptor protocol defines how attribute access is interpreted by the language. A descriptor class can provide `__get__` and `__set__` methods that let you reuse the grade validation behavior without any boilerplate. For this purpose, descriptors are also better than mix-ins (see [Item 26: "Use Multiple Inheritance Only for Mix-in Utility Classes"](#)) because they let you reuse the same logic for many different attributes in a single class.

Here, I define a new class called `Exam` with class attributes that are `Grade` instances. The `Grade` class implements the descriptor protocol. Before I explain how the `Grade` class works, it's important to understand what Python will do when your code accesses such descriptor attributes on an `Exam` instance.

[Click here to view code image](#)

```

class Grade(object):
    def __get__(*args, **kwargs):
        # ...

    def __set__(*args, **kwargs):
        # ...

class Exam(object):

```

```
# Class attributes
math_grade = Grade()
writing_grade = Grade()
science_grade = Grade()
```

When you assign a property:

```
exam = Exam()
exam.writing_grade = 40
```

it will be interpreted as:

[Click here to view code image](#)

```
Exam.__dict__['writing_grade'].__set__(exam, 40)
```

When you retrieve a property:

```
print(exam.writing_grade)
```

it will be interpreted as:

[Click here to view code image](#)

```
print(Exam.__dict__['writing_grade'].__get__(exam, Exam))
```

What drives this behavior is the `__getattribute__` method of object (see [Item 32: “Use `\_\_getattr\_\_`, `\_\_getattribute\_\_`, and `\_\_setattr\_\_` for Lazy Attributes](#)”). In short, when an `Exam` instance doesn't have an attribute named `writing_grade`, Python will fall back to the `Exam` class's attribute instead. If this class attribute is an object that has `__get__` and `__set__` methods, Python will assume you want to follow the descriptor protocol.

Knowing this behavior and how I used `@property` for grade validation in the `Homework` class, here's a reasonable first attempt at implementing the `Grade` descriptor.

[Click here to view code image](#)

```
class Grade(object):
    def __init__(self):
        self._value = 0

    def __get__(self, instance, instance_type):
        return self._value

    def __set__(self, instance, value):
        if not (0 <= value <= 100):
            raise ValueError('Grade must be between 0 and 100')
        self._value = value
```

Unfortunately, this is wrong and will result in broken behavior. Accessing multiple attributes on a single `Exam` instance works as expected.

[Click here to view code image](#)

```
first_exam = Exam()
first_exam.writing_grade = 82
first_exam.science_grade = 99
print('Writing', first_exam.writing_grade)
print('Science', first_exam.science_grade)

>>>
Writing 82
```

But accessing these attributes on multiple `Exam` instances will have unexpected behavior.

[Click here to view code image](#)

```
second_exam = Exam()
second_exam.writing_grade = 75
print('Second', second_exam.writing_grade, 'is right')
print('First ', first_exam.writing_grade, 'is wrong')

>>>
Second 75 is right
First 75 is wrong
```

The problem is that a single `Grade` instance is shared across all `Exam` instances for the class attribute `writing_grade`. The `Grade` instance for this attribute is constructed once in the program lifetime when the `Exam` class is first defined, not each time an `Exam` instance is created.

To solve this, I need the `Grade` class to keep track of its value for each unique `Exam` instance. I can do this by saving the per-instance state in a dictionary.

[Click here to view code image](#)

```
class Grade(object):
    def __init__(self):
        self._values = {}

    def __get__(self, instance, instance_type):
        if instance is None: return self
        return self._values.get(instance, 0)

    def __set__(self, instance, value):
        if not (0 <= value <= 100):
            raise ValueError('Grade must be between 0 and 100')
        self._values[instance] = value
```

This implementation is simple and works well, but there's still one gotcha: It leaks memory. The `_values` dictionary will hold a reference to every instance of `Exam` ever passed to `__set__` over the lifetime of the program. This causes instances to never have their reference count go to zero, preventing cleanup by the garbage collector.

To fix this, I can use Python's `weakref` built-in module. This module provides a special class called `WeakKeyDictionary` that can take the place of the simple dictionary used for `_values`. The unique behavior of `WeakKeyDictionary` is that it will remove `Exam` instances from its set of keys when the runtime knows it's holding the instance's last remaining reference in the program. Python will do the bookkeeping for you and ensure that the `_values` dictionary will be empty when all `Exam` instances are no longer in use.

[Click here to view code image](#)

```
class Grade(object):
    def __init__(self):
        self._values = WeakKeyDictionary()
    # ...
```

Using this implementation of the `Grade` descriptor, everything works as expected.

[Click here to view code image](#)

```
class Exam(object):
    math_grade = Grade()
    writing_grade = Grade()
    science_grade = Grade()
first_exam = Exam()
first_exam.writing_grade = 82
second_exam = Exam()
second_exam.writing_grade = 75
print('First ', first_exam.writing_grade, 'is right')
print('Second', second_exam.writing_grade, 'is right')

>>>
First 82 is right
Second 75 is right
```

Things to Remember

- ◆ Reuse the behavior and validation of `@property` methods by defining your own descriptor classes.
- ◆ Use `WeakKeyDictionary` to ensure that your descriptor classes don't cause memory leaks.
- ◆ Don't get bogged down trying to understand exactly how `__getattr__` uses the descriptor protocol for getting and setting attributes.

Item 32: Use `__getattr__`, `__getattr__`, and `__setattr__` for Lazy Attributes

Python's language hooks make it easy to write generic code for gluing systems together. For example, say you want to represent the rows of your database as Python objects. Your database has its schema set. Your code that uses objects corresponding to those rows must also know what your database looks like. However, in Python, the code that connects your Python objects to the database doesn't need to know the schema of your rows; it can be generic.

How is that possible? Plain instance attributes, `@property` methods, and descriptors can't do this because they all need to be defined in advance. Python makes this dynamic behavior possible with the `__getattr__` special method. If your class defines `__getattr__`, that method is called every time an attribute can't be found in an object's instance dictionary.

[Click here to view code image](#)

```
class LazyDB(object):
    def __init__(self):
        self.exists = 5

    def __getattr__(self, name):
        value = 'Value for %s' % name
        setattr(self, name, value)
        return value
```

Here, I access the missing property `foo`. This causes Python to call the `__getattr__` method above, which mutates the instance dictionary `__dict__`:

[Click here to view code image](#)

```
data = LazyDB()
print('Before:', data.__dict__)
print('foo: ', data.foo)
print('After: ', data.__dict__)

>>>
Before: {'exists': 5}
foo:    Value for foo
After:  {'exists': 5, 'foo': 'Value for foo'}
```

Here, I add logging to `LazyDB` to show when `__getattr__` is actually called. Note that I use `super().__getattr__()` to get the real property value in order to avoid infinite recursion.

[Click here to view code image](#)

```
class LoggingLazyDB(LazyDB):
    def __getattr__(self, name):
        print('Called __getattr__(%s)' % name)
        return super().__getattr__(name)

data = LoggingLazyDB()
print('exists:', data.exists)
print('foo: ', data.foo)
print('foo: ', data.foo)

>>>
exists: 5
Called __getattr__(foo)
foo:    Value for foo
foo:    Value for foo
```

The `exists` attribute is present in the instance dictionary, so `__getattr__` is never called for it. The `foo` attribute is not in the instance dictionary initially, so `__getattr__` is called the first time. But the call to `__getattr__` for `foo` also does a `setattr`, which populates `foo` in the instance dictionary. This is why the second time I access `foo` there isn't a call to `__getattr__`.

This behavior is especially helpful for use cases like lazily accessing schemaless data. `__getattr__` runs once to do the hard work of loading a property; all subsequent accesses retrieve the existing result.

Say you also want transactions in this database system. The next time the user accesses a property, you want to know whether the corresponding row in the database is still valid and whether the transaction is still open. The `__getattr__` hook won't let you do this reliably because it will use the object's instance dictionary as the fast path for existing attributes.

To enable this use case, Python has another language hook called `__getattribute__`. This special method is called every time an attribute is accessed on an object, even in cases where it *does* exist in the attribute dictionary. This enables you to do things like

check global transaction state on every property access. Here, I define `ValidatingDB` to log each time `__getattr__` is called:

[Click here to view code image](#)

```
class ValidatingDB(object):
    def __init__(self):
        self.exists = 5

    def __getattr__(self, name):
        print('Called __getattr__((%s)' % name)
        try:
            return super().__getattr__(name)
        except AttributeError:
            value = 'Value for %s' % name
            setattr(self, name, value)
            return value

data = ValidatingDB()
print('exists:', data.exists)
print('foo: ', data.foo)
print('foo: ', data.foo)

>>>
Called __getattr__(exists)
exists: 5
Called __getattr__(foo)
foo: Value for foo
Called __getattr__(foo)
foo: Value for foo
```

In the event that a dynamically accessed property shouldn't exist, you can raise an `AttributeError` to cause Python's standard missing property behavior for both `__getattr__` and `__getattribute__`.

[Click here to view code image](#)

```
class MissingPropertyDB(object):
    def __getattr__(self, name):
        if name == 'bad_name':
            raise AttributeError('%s is missing' % name)
        # ...

data = MissingPropertyDB()
data.bad_name

>>>
AttributeError: bad_name is missing
```

Python code implementing generic functionality often relies on the `hasattr` built-in function to determine when properties exist, and the `getattr` built-in function to retrieve property values. These functions also look in the instance dictionary for an attribute name before calling `__getattr__`.

[Click here to view code image](#)

```
data = LoggingLazyDB()
print('Before: ', data.__dict__)
print('foo exists: ', hasattr(data, 'foo'))
print('After: ', data.__dict__)
```

```
print('foo exists: ', hasattr(data, 'foo'))

>>>
Before:      {'exists': 5}
Called __getattr__(foo)
foo exists:  True
After:       {'exists': 5, 'foo': 'Value for foo'}
foo exists:  True
```

In the example above, `__getattr__` is only called once. In contrast, classes that implement `__getattribute__` will have that method called each time `hasattr` or `getattr` is run on an object.

[Click here to view code image](#)

```
data = ValidatingDB()
print('foo exists: ', hasattr(data, 'foo'))
print('foo exists: ', hasattr(data, 'foo'))

>>>
Called __getattribute__(foo)
foo exists:  True
Called __getattribute__(foo)
foo exists:  True
```

Now, say you want to lazily push data back to the database when values are assigned to your Python object. You can do this with `__setattr__`, a similar language hook that lets you intercept arbitrary attribute assignments. Unlike retrieving an attribute with `__getattr__` and `__getattribute__`, there's no need for two separate methods. The `__setattr__` method is always called every time an attribute is assigned on an instance (either directly or through the `setattr` built-in function).

[Click here to view code image](#)

```
class SavingDB(object):
    def __setattr__(self, name, value):
        # Save some data to the DB log
        # ...
        super().__setattr__(name, value)
```

Here, I define a logging subclass of `SavingDB`. Its `__setattr__` method is always called on each attribute assignment:

[Click here to view code image](#)

```
class LoggingSavingDB(SavingDB):
    def __setattr__(self, name, value):
        print('Called __setattr__(%s, %r)' % (name, value))
        super().__setattr__(name, value)

data = LoggingSavingDB()
print('Before: ', data.__dict__)
data.foo = 5
print('After: ', data.__dict__)
data.foo = 7
print('Finally:', data.__dict__)

>>>
Before:  {}
Called __setattr__(foo, 5)
```

```
After:    {'foo': 5}
Called __setattr__(foo, 7)
Finally:  {'foo': 7}
```

The problem with `__getattribute__` and `__setattr__` is that they're called on every attribute access for an object, even when you may not want that to happen. For example, say you want attribute accesses on your object to actually look up keys in an associated dictionary.

[Click here to view code image](#)

```
class BrokenDictionaryDB(object):
    def __init__(self, data):
        self._data = {}

    def __getattribute__(self, name):
        print('Called __getattribute__(%s)' % name)
        return self._data[name]
```

This requires accessing `self._data` from the `__getattribute__` method. However, if you actually try to do that, Python will recurse until it reaches its stack limit, and then it'll die.

[Click here to view code image](#)

```
data = BrokenDictionaryDB({'foo': 3})
data.foo

>>>
Called __getattribute__(foo)
Called __getattribute__(_data)
Called __getattribute__(_data)
...
Traceback ...
RuntimeError: maximum recursion depth exceeded
```

The problem is that `__getattribute__` accesses `self._data`, which causes `__getattribute__` to run again, which accesses `self._data` again, and so on. The solution is to use the `super().__getattribute__` method on your instance to fetch values from the instance attribute dictionary. This avoids the recursion.

[Click here to view code image](#)

```
class DictionaryDB(object):
    def __init__(self, data):
        self._data = data

    def __getattribute__(self, name):
        data_dict = super().__getattribute__('_data')
        return data_dict[name]
```

Similarly, you'll need `__setattr__` methods that modify attributes on an object to use `super().__setattr__`.

Things to Remember

- ◆ Use `__getattr__` and `__setattr__` to lazily load and save attributes for an object.

- ◆ Understand that `__getattr__` only gets called once when accessing a missing attribute, whereas `__getattribute__` gets called every time an attribute is accessed.
- ◆ Avoid infinite recursion in `__getattribute__` and `__setattr__` by using methods from `super()` (i.e., the object class) to access instance attributes directly.

Item 33: Validate Subclasses with Metaclasses

One of the simplest applications of metaclasses is verifying that a class was defined correctly. When you're building a complex class hierarchy, you may want to enforce style, require overriding methods, or have strict relationships between class attributes. Metaclasses enable these use cases by providing a reliable way to run your validation code each time a new subclass is defined.

Often a class's validation code runs in the `__init__` method, when an object of the class's type is constructed (see [Item 28: “Inherit from `collections.abc` for Custom Container Types](#)” for an example). Using metaclasses for validation can raise errors much earlier.

Before I get into how to define a metaclass for validating subclasses, it's important to understand the metaclass action for standard objects. A metaclass is defined by inheriting from `type`. In the default case, a metaclass receives the contents of associated `class` statements in its `__new__` method. Here, you can modify the class information before the type is actually constructed:

[Click here to view code image](#)

```
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        print((meta, name, bases, class_dict))
        return type.__new__(meta, name, bases, class_dict)

class MyClass(object, metaclass=Meta):
    stuff = 123

    def foo(self):
        pass
```

The metaclass has access to the name of the class, the parent classes it inherits from, and all of the class attributes that were defined in the `class`'s body.

[Click here to view code image](#)

```
>>>
(<class '__main__.Meta'>,
 'MyClass',
 (<class 'object'>,),
 {'__module__': '__main__',
  '__qualname__': 'MyClass',
  'foo': <function MyClass.foo at 0x102c7dd08>,
  'stuff': 123})
```

Python 2 has slightly different syntax and specifies a metaclass using the `__metaclass__` class attribute. The `Meta.__new__` interface is the same.

[Click here to view code image](#)

```
# Python 2
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        # ...
class MyClassInPython2(object):
    __metaclass__ = Meta
    # ...
```

You can add functionality to the `Meta.__new__` method in order to validate all of the parameters of a class before it's defined. For example, say you want to represent any type of multisided polygon. You can do this by defining a special validating metaclass and using it in the base class of your polygon class hierarchy. Note that it's important not to apply the same validation to the base class.

[Click here to view code image](#)

```
class ValidatePolygon(type):
    def __new__(meta, name, bases, class_dict):
        # Don't validate the abstract Polygon class
        if bases != (object,):
            if class_dict['sides'] < 3:
                raise ValueError('Polygons need 3+ sides')
            return type.__new__(meta, name, bases, class_dict)

class Polygon(object, metaclass=ValidatePolygon):
    sides = None # Specified by subclasses

    @classmethod
    def interior_angles(cls):
        return (cls.sides - 2) * 180

class Triangle(Polygon):
    sides = 3
```

If you try to define a polygon with fewer than three sides, the validation will cause the `class` statement to fail immediately after the `class` statement body. This means your program will not even be able to start running when you define such a class.

[Click here to view code image](#)

```
print('Before class')
class Line(Polygon):
    print('Before sides')
    sides = 1
    print('After sides')
print('After class')
```

```
>>>
Before class
Before sides
After sides
Traceback ...
ValueError: Polygons need 3+ sides
```

Things to Remember

- ◆ Use metaclasses to ensure that subclasses are well formed at the time they are defined, before objects of their type are constructed.
- ◆ Metaclasses have slightly different syntax in Python 2 vs. Python 3.
- ◆ The `__new__` method of metaclasses is run after the `class` statement's entire body has been processed.

Item 34: Register Class Existence with Metaclasses

Another common use of metaclasses is to automatically register types in your program. Registration is useful for doing reverse lookups, where you need to map a simple identifier back to a corresponding class.

For example, say you want to implement your own serialized representation of a Python object using JSON. You need a way to take an object and turn it into a JSON string. Here, I do this generically by defining a base class that records the constructor parameters and turns them into a JSON dictionary:

[Click here to view code image](#)

```
class Serializable(object):
    def __init__(self, *args):
        self.args = args

    def serialize(self):
        return json.dumps({'args': self.args})
```

This class makes it easy to serialize simple, immutable data structures like `Point2D` to a string.

[Click here to view code image](#)

```
class Point2D(Serializable):
    def __init__(self, x, y):
        super().__init__(x, y)
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Point2D(%d, %d)' % (self.x, self.y)
```

```
point = Point2D(5, 3)
print('Object: ', point)
print('Serialized:', point.serialize())
```

```
>>>
Object:      Point2D(5, 3)
Serialized: {"args": [5, 3]}
```

Now, I need to deserialize this JSON string and construct the `Point2D` object it represents. Here, I define another class that can deserialize the data from its `Serializable` parent class:

[Click here to view code image](#)

```

class Deserializable(Serializable):
    @classmethod
    def deserialize(cls, json_data):
        params = json.loads(json_data)
        return cls(*params['args'])

```

Using `Deserializable` makes it easy to serialize and deserialize simple, immutable objects in a generic way.

[Click here to view code image](#)

```

class BetterPoint2D(Deserializable):
    # ...
point = BetterPoint2D(5, 3)
print('Before: ', point)
data = point.serialize()
print('Serialized:', data)
after = BetterPoint2D.deserialize(data)
print('After: ', after)

>>>
Before:      BetterPoint2D(5, 3)
Serialized:  {"args": [5, 3]}
After:       BetterPoint2D(5, 3)

```

The problem with this approach is that it only works if you know the intended type of the serialized data ahead of time (e.g., `Point2D`, `BetterPoint2D`). Ideally, you'd have a large number of classes serializing to JSON and one common function that could deserialize any of them back to a corresponding Python object.

To do this, I can include the serialized object's class name in the JSON data.

[Click here to view code image](#)

```

class BetterSerializable(object):
    def __init__(self, *args):
        self.args = args

    def serialize(self):
        return json.dumps({
            'class': self.__class__.__name__,
            'args': self.args,
        })
    def __repr__(self):
        # ...

```

Then, I can maintain a mapping of class names back to constructors for those objects. The general `deserialize` function will work for any classes passed to `register_class`.

[Click here to view code image](#)

```

registry = {}

def register_class(target_class):
    registry[target_class.__name__] = target_class

def deserialize(data):
    params = json.loads(data)
    name = params['class']
    target_class = registry[name]

```

```
return target_class(*params['args'])
```

To ensure that `deserialize` always works properly, I must call `register_class` for every class I may want to deserialize in the future.

[Click here to view code image](#)

```
class EvenBetterPoint2D(BetterSerializable):
    def __init__(self, x, y):
        super().__init__(x, y)
        self.x = x
        self.y = y

register_class(EvenBetterPoint2D)
```

Now, I can deserialize an arbitrary JSON string without having to know which class it contains.

[Click here to view code image](#)

```
point = EvenBetterPoint2D(5, 3)
print('Before: ', point)
data = point.serialize()
print('Serialized:', data)
after = deserialize(data)
print('After: ', after)

>>>
Before:      EvenBetterPoint2D(5, 3)
Serialized:  {"class": "EvenBetterPoint2D", "args": [5, 3]}
After:      EvenBetterPoint2D(5, 3)
```

The problem with this approach is that you can forget to call `register_class`.

[Click here to view code image](#)

```
class Point3D(BetterSerializable):
    def __init__(self, x, y, z):
        super().__init__(x, y, z)
        self.x = x
        self.y = y
        self.z = z

# Forgot to call register_class! Whoops!
```

This will cause your code to break at runtime, when you finally try to deserialize an object of a class you forgot to register.

```
point = Point3D(5, 9, -4)
data = point.serialize()
deserialize(data)

>>>
KeyError: 'Point3D'
```

Even though you chose to subclass `BetterSerializable`, you won't actually get all of its features if you forget to call `register_class` after your `class` statement body. This approach is error prone and especially challenging for beginners. The same omission can happen with *class decorators* in Python 3.

What if you could somehow act on the programmer's intent to use

`BetterSerializable` and ensure that `register_class` is called in all cases? Metaclasses enable this by intercepting the `class` statement when subclasses are defined (see [Item 33: “Validate Subclasses with Metaclasses”](#)). This lets you register the new type immediately after the class’s body.

[Click here to view code image](#)

```
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        cls = type.__new__(meta, name, bases, class_dict)
        register_class(cls)
        return cls
class RegisteredSerializable(BetterSerializable,
                             metaclass=Meta):
    pass
```

When I define a subclass of `RegisteredSerializable`, I can be confident that the call to `register_class` happened and `deserialize` will always work as expected.

[Click here to view code image](#)

```
class Vector3D(RegisteredSerializable):
    def __init__(self, x, y, z):
        super().__init__(x, y, z)
        self.x, self.y, self.z = x, y, z

v3 = Vector3D(10, -7, 3)
print('Before:      ', v3)
data = v3.serialize()
print('Serialized:', data)
print('After:       ', deserialize(data))

>>>
Before:      Vector3D(10, -7, 3)
Serialized:  {"class": "Vector3D", "args": [10, -7, 3]}
After:       Vector3D(10, -7, 3)
```

Using metaclasses for class registration ensures that you’ll never miss a class as long as the inheritance tree is right. This works well for serialization, as I’ve shown, and also applies to database object-relationship mappings (ORMs), plug-in systems, and system hooks.

Things to Remember

- ◆ Class registration is a helpful pattern for building modular Python programs.
- ◆ Metaclasses let you run registration code automatically each time your base class is subclassed in a program.
- ◆ Using metaclasses for class registration avoids errors by ensuring that you never miss a registration call.

Item 35: Annotate Class Attributes with Metaclasses

One more useful feature enabled by metaclasses is the ability to modify or annotate properties after a class is defined but before the class is actually used. This approach is commonly used with *descriptors* (see [Item 31: “Use Descriptors for Reusable @property Methods”](#)) to give them more introspection into how they’re being used within their containing class.

For example, say you want to define a new class that represents a row in your customer database. You’d like a corresponding property on the class for each column in the database table. To do this, here I define a descriptor class to connect attributes to column names.

[Click here to view code image](#)

```
class Field(object):
    def __init__(self, name):
        self.name = name
        self.internal_name = '_' + self.name

    def __get__(self, instance, instance_type):
        if instance is None: return self
        return getattr(instance, self.internal_name, "")

    def __set__(self, instance, value):
        setattr(instance, self.internal_name, value)
```

With the column name stored in the `Field` descriptor, I can save all of the per-instance state directly in the instance dictionary as protected fields using the `setattr` and `getattr` built-in functions. At first, this seems to be much more convenient than building descriptors with `weakref` to avoid memory leaks.

Defining the class representing a row requires supplying the column name for each class attribute.

[Click here to view code image](#)

```
class Customer(object):
    # Class attributes
    first_name = Field('first_name')
    last_name = Field('last_name')
    prefix = Field('prefix')
    suffix = Field('suffix')
```

Using the class is simple. Here, you can see how the `Field` descriptors modify the instance dictionary `__dict__` as expected:

[Click here to view code image](#)

```
foo = Customer()
print('Before:', repr(foo.first_name), foo.__dict__)
foo.first_name = 'Euclid'
print('After: ', repr(foo.first_name), foo.__dict__)

>>>
Before: " {}
After:  'Euclid' {'_first_name': 'Euclid'}
```

But it seems redundant. I already declared the name of the field when I assigned the

constructed `Field` object to `Customer.first_name` in the `class` statement body. Why do I also have to pass the field name (`'first_name'` in this case) to the `Field` constructor?

The problem is that the order of operations in the `Customer` class definition is the opposite of how it reads from left to right. First, the `Field` constructor is called as `Field('first_name')`. Then, the return value of that is assigned to `Customer.field_name`. There's no way for the `Field` to know upfront which class attribute it will be assigned to.

To eliminate the redundancy, I can use a metaclass. Metaclasses let you hook the `class` statement directly and take action as soon as a `class` body is finished. In this case, I can use the metaclass to assign `Field.name` and `Field.internal_name` on the descriptor automatically instead of manually specifying the field name multiple times.

[Click here to view code image](#)

```
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        for key, value in class_dict.items():
            if isinstance(value, Field):
                value.name = key
                value.internal_name = '_' + key
        cls = type.__new__(meta, name, bases, class_dict)
        return cls
```

Here, I define a base class that uses the metaclass. All classes representing database rows should inherit from this class to ensure that they use the metaclass:

[Click here to view code image](#)

```
class DatabaseRow(object, metaclass=Meta):
    pass
```

To work with the metaclass, the field descriptor is largely unchanged. The only difference is that it no longer requires any arguments to be passed to its constructor. Instead, its attributes are set by the `Meta.__new__` method above.

[Click here to view code image](#)

```
class Field(object):
    def __init__(self):
        # These will be assigned by the metaclass.
        self.name = None
        self.internal_name = None
    # ...
```

By using the metaclass, the new `DatabaseRow` base class, and the new `Field` descriptor, the class definition for a database row no longer has the redundancy from before.

[Click here to view code image](#)

```
class BetterCustomer(DatabaseRow):
    first_name = Field()
    last_name = Field()
    prefix = Field()
    suffix = Field()
```

The behavior of the new class is identical to the old one.

[Click here to view code image](#)

```
foo = BetterCustomer()
print('Before:', repr(foo.first_name), foo.__dict__)
foo.first_name = 'Euler'
print('After: ', repr(foo.first_name), foo.__dict__)

>>>
Before: " {}
After:  'Euler' {'_first_name': 'Euler'}
```

Things to Remember

- ◆ Metaclasses enable you to modify a class's attributes before the class is fully defined.
- ◆ Descriptors and metaclasses make a powerful combination for declarative behavior and runtime introspection.
- ◆ You can avoid both memory leaks and the `weakref` module by using metaclasses along with descriptors.